

REAL-TIME AIRFARE PRICE INDEX FOR INDIA

Product Requirements Document (PRD) + Technical Design Document (TDD)

Internal Hackathon Prototype

Project	Automated Airfare Price Index Platform
Primary Goal	Convert permitted online airfare observations into a daily/weekly/monthly weighted index
Prototype Scope	6-10 routes, 3-5 sources, T+1/T+7/T+15/T+30/T+45, 30-day back-test
Frontend	React + TypeScript + Vite + Tailwind CSS
Backend	Python + FastAPI
Database	MongoDB Atlas
Collection	Playwright / Scrapy / authorized APIs where available
Deployment	Vercel (frontend) + Render (backend)

Important scope note: This document defines a prototype for an internal hackathon. It is not an official CPI methodology or an official NSO/RBI system. Any source that prohibits automated collection must be handled through an authorized interface, permitted public data, or omitted; the prototype must not bypass CAPTCHA, access controls, or terms of service.

PART I — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Executive Summary

The project is an end-to-end software platform that collects permitted airfare observations from airline/online travel sources, standardizes the observations, stores them with rich metadata, cleans and validates the data, and computes an experimental Real-time Airfare Price Index (APIx) for India. The dashboard provides index trends, route comparisons, lead-time analysis, data-quality metrics, and a REST API for downstream analytical use.

2. Problem Statement

Traditional/manual airfare price collection is not well suited to highly dynamic online airfare pricing. The prototype addresses this by capturing repeated, route-specific price observations across multiple advance-purchase windows and transforming them into a transparent weighted index.

3. Objectives

- Automate permitted collection/import of airfare quotes from multiple sources.
- Maintain a clean, de-duplicated database of route, airline, date, fare and source metadata.
- Capture T+1, T+7, T+15, T+30 and T+45 advance-purchase windows.
- Construct a transparent experimental weighted Airfare Price Index with a documented base period.
- Provide daily, weekly and monthly index views.
- Visualize route-level prices, heatmaps, lead-time curves and anomalies.
- Expose the index through a documented REST API.
- Demonstrate 30 days of back-tested/synthetic validation against a legitimate reference dataset where available.

4. Target Users

User	Need
Hackathon evaluator	Understand the end-to-end solution and see measurable output.
Economic/data analyst	Inspect index methodology, route trends and price components.
System administrator	Monitor collection jobs and data quality.
Future institutional consumer	Consume standardized index data through an API.

5. Prototype Scope

Area	Prototype Target
Routes	6–10 representative domestic city-pairs
Sources	3–5 permitted/authorized sources or clearly labelled prepared datasets
Airlines	Major carriers represented where permitted
Advance windows	T+1, T+7, T+15, T+30, T+45
History	30-day back-test/demo dataset
Index frequency	Daily; weekly/monthly aggregation
Dashboard	Overview, index trend, route analysis, heatmap, lead-time, data quality
API	Current index, history, routes, route details, analytics, data quality

6. Out of Scope for the Hackathon

- Nationwide production-scale coverage of every Indian route.

- Guaranteed live access to every airline/OTA.
- Bypassing CAPTCHA, bot protection, login restrictions or other access controls.
- Official integration into NSO/RBI production systems.
- Claiming the prototype is an official CPI component.
- Complex deep-learning airfare forecasting unless time permits.

7. Functional Requirements

ID	Requirement	Priority
FR-01	Store route, origin, destination, airline, source and timestamp metadata.	Must
FR-02	Store travel date and advance-purchase window.	Must
FR-03	Store base fare, taxes, fees and total fare separately when available.	Must
FR-04	Validate and normalize collected/imported quotes.	Must
FR-05	Detect duplicates and flag potential outliers.	Must
FR-06	Handle missing, cancelled and sold-out observations according to documented rules.	Must
FR-07	Calculate route-level price relatives and weighted composite index.	Must
FR-08	Provide daily, weekly and monthly index values.	Must
FR-09	Display historical trends and route comparisons.	Must
FR-10	Display T+1 through T+45 lead-time analysis.	Should
FR-11	Expose index and analytics through REST endpoints.	Must
FR-12	Record collection-run and data-quality status.	Should
FR-13	Provide 30-day validation/back-test view.	Must

8. Non-Functional Requirements

- Reliability: failed source jobs must be logged without stopping the entire pipeline.
- Performance: dashboard API responses should normally return within 2 seconds for prototype-scale queries.
- Security: secrets stored in environment variables; no credentials committed to Git.
- Maintainability: each data source should use an adapter/collector interface.
- Traceability: every quote must retain source and collection timestamp.
- Scalability: design should allow more routes and sources without rewriting the index engine.
- Ethics/compliance: respect source terms, robots rules where applicable, rate limits and access controls.

9. Core User Stories

- As an analyst, I want to see the current Airfare Price Index so I can understand the latest movement.
- As an analyst, I want to select a route and advance window so I can inspect its fare behavior.
- As a data engineer, I want to inspect invalid/outlier records so I can monitor data quality.
- As a reviewer, I want to see the base period, route basket and weights so the index is transparent.
- As an API consumer, I want machine-readable index history so I can use the data in another application.

10. Success Criteria

Metric	Target
Routes represented	≥ 6

Metric	Target
Advance windows	5
Sources	≥ 3 permitted/authorized sources or labelled prepared datasets
Database records	Enough to demonstrate multi-day, multi-route analysis
Index	Daily index plus weekly/monthly aggregation
Dashboard	All core visualizations functional
API	Core endpoints return valid JSON
Validation	30-day back-test/demo with documented reference/synthetic status
Testing	Unit + API + data-pipeline smoke tests

PART II — TECHNICAL DESIGN DOCUMENT (TDD)

11. System Architecture

The architecture separates data collection, storage, data quality, index calculation, API delivery and visualization. This makes the prototype easier to test and scale.

Architecture flow

Sources → Collector Adapters → Scheduler → Raw/Normalized MongoDB → Cleaning & Validation → Index Engine → FastAPI → React Dashboard

12. Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Dashboard and user interaction
UI	Tailwind CSS	Responsive interface
Charts	Recharts	Index and lead-time visualizations
Maps	Leaflet	Route visualization
Backend	Python + FastAPI	REST API and business logic
Collection	Playwright / Scrapy / authorized APIs	Permitted data acquisition
Processing	Pandas + NumPy + SciPy	Cleaning and statistical processing
ML/Anomaly	Scikit-learn	Optional anomaly detection
Database	MongoDB Atlas	Airfare and index storage
Scheduler	APScheduler	Scheduled collection jobs
Deployment	Vercel + Render	Prototype hosting
Version control	GitHub	Source control and collaboration

13. Data Model

Primary collection: airfare_quotes

```
{ "route_id": "MAA-DEL", "origin": "MAA", "destination": "DEL", "airline": "Example Airline", "travel_date": "2026-09-15",  
  "collected_at": "2026-08-26T10:00:00", "advance_days": 20, "cabin": "Economy", "fare_class": "standard", "base_fare":  
  4200, "taxes": 700, "fees": 80, "total_fare": 4980, "currency": "INR", "availability": true, "source": "permitted_source",  
  "status": "valid" }
```

Supporting collections

Collection	Key fields
routes	route_id, origin, destination, weight, active
airlines	airline_id, name, active
index_values	date, frequency, index_value, base_period, methodology_version
collection_runs	source, started_at, completed_at, status, records_found, errors
data_quality	date, valid_count, missing_count, duplicate_count, outlier_count

14. Data Collection Design

- Use a common Collector interface so each source has an independent adapter.
- Prefer authorized APIs or permitted public collection where available.
- Use Playwright only where permitted and necessary for JavaScript-rendered pages.
- Apply conservative rate limits and retries with exponential backoff.

- Log source failures, timeouts and parsing errors.
- Never bypass CAPTCHA, authentication controls or anti-bot protections.
- Store collection timestamp and source identifier for every observation.

15. Cleaning & Normalization Pipeline

Raw quote → schema validation → type normalization → duplicate detection → missing-value rules → outlier flagging → fare-component normalization → validity status → index-ready quote.

Suggested outlier approach: use route/window-specific robust statistics such as median and MAD/IQR. Do not silently delete extreme observations; retain them with a quality flag and document the treatment used for the index.

16. Index Methodology

For the prototype, define a fixed base period and route basket. Let $P(i,t)$ be the standardized price for route i at time t , $P(i,0)$ the corresponding base-period price, and $w(i)$ the route weight with $\sum w(i)=1$.

Experimental weighted price-relative index:

$$I(t) = 100 \times \sum [w(i) \times (P(i,t) / P(i,0))]$$

The exact basket and weights must be documented and versioned. For the hackathon, weights can be based on a clearly cited DGCA traffic dataset or a transparent prototype weighting scheme if the official dataset is not available. The result must be labelled experimental.

17. Frequency Aggregation

- Daily: calculate the index from valid observations collected for the day.
- Weekly: aggregate daily index values using a documented method such as arithmetic mean.
- Monthly: aggregate daily values for the month and preserve the methodology version.
- If a required route has no valid observation, apply the documented missing-data rule rather than inventing a value.

18. API Design

Method	Endpoint	Purpose
GET	/api/index/current	Latest index and change
GET	/api/index/history	Historical index series
GET	/api/index/daily	Daily index values
GET	/api/index/weekly	Weekly index values
GET	/api/index/monthly	Monthly index values
GET	/api/routes	Route basket and weights
GET	/api/routes/{route}	Route-level price/index data
GET	/api/airlines	Airline list and observations
GET	/api/analytics/lead-time	T+1 to T+45 analysis
GET	/api/data-quality	Collection and quality metrics

19. Frontend Design

- Overview: current index, daily/monthly change, route count, observation count and data quality.
- Index Trend: interactive time-series chart.
- Route Analysis: fare, index and history for a selected city-pair.
- Route Heatmap: route-level index changes.
- Lead-Time Analysis: T+1/T+7/T+15/T+30/T+45 comparison.

- Data Quality: valid, missing, duplicate and outlier counts.
- Methodology: base period, basket, weights and methodology version.

20. Anomaly Detection

Optional but recommended for the hackathon: calculate a rolling median/mean and dispersion for each route and advance window. Flag observations that exceed a configurable threshold. Isolation Forest can be added later if time permits.

21. Testing Strategy

Test Type	Examples
Unit tests	Index formula, weighting, normalization, outlier flagging
API tests	Endpoint status codes, schema, filtering
Data tests	Required fields, valid INR values, route codes
Integration	Collector → database → cleaning → index
Frontend	Dashboard rendering, route filters, empty/error states
Smoke test	Run one complete pipeline end-to-end

22. 30-Day Back-Test Plan

- Prepare or collect a legitimate 30-day observation/reference dataset.
- Freeze a methodology version and base period before evaluation.
- Calculate the prototype index for each day.
- Compare the index against the reference series where the definitions are comparable.
- Report MAE, RMSE, correlation and percentage deviation.
- Clearly distinguish real observations, backfilled observations and synthetic demonstration data.

23. Deployment Architecture

Browser → Vercel React application → Render FastAPI service → MongoDB Atlas. Scheduled collection can run as a background service/worker on the backend host or through a suitable scheduler. Environment variables hold MongoDB URI, API secrets and other configuration values.

24. Security

- Use environment variables for secrets.
- Enable MongoDB network access controls appropriate to the deployment.
- Validate all API parameters.
- Apply API rate limiting if publicly exposed.
- Do not store unnecessary personal information.
- Log errors without logging credentials or sensitive tokens.

25. Suggested Repository Structure

```
airfare-index/ ├── frontend/ | ├── src/components/ | ├── src/pages/ | ├── src/charts/ | ├── src/services/ | └── src/types/ |
backend/ | ├── app/main.py | ├── api/ | ├── collectors/ | ├── processors/ | ├── index_engine/ | ├── models/ | ├──
database/ | └── tests/ | ├── data/ | ├── sample/ | └── reference/ | ├── docs/ | └── README.md
```

26. Hackathon Implementation Roadmap

Phase	Deliverable
1	Project setup, MongoDB schema and sample dataset
2	Cleaning/normalization pipeline
3	Index engine with route weights and base period
4	FastAPI endpoints
5	React dashboard and charts
6	Permitted source collector/demo import
7	30-day back-test and data-quality page
8	Testing, deployment and final demo

27. Final Demo Checklist

- Show a route basket and weights.
- Show raw-to-cleaned data transformation.
- Show T+1 to T+45 fare observations.
- Show current Airfare Price Index.
- Show daily/weekly/monthly trend.
- Show route heatmap.
- Show data-quality metrics.
- Show 30-day validation/back-test.
- Call the REST API and display JSON.
- Explain ethical data collection and limitations.

28. Risks and Mitigations

Risk	Mitigation
Source blocks automation	Use authorized API/permitted data or exclude source; never bypass access controls.
Website structure changes	Independent adapters + parser tests.
Missing prices	Documented imputation/omission rules and quality flags.
Extreme fares	Robust outlier detection with audit trail.
Insufficient historical data	Use clearly labelled backfilled/public/synthetic demonstration data.
Index methodology challenged	Publish base period, basket, weights and formula.
Deployment failure	Keep local demo dataset and reproducible seed data.
Slow dashboard	Pre-aggregate index data and index MongoDB query fields.

29. Definition of Done

- The prototype can ingest/import airfare observations into MongoDB.
- The cleaning pipeline produces index-ready observations with quality flags.
- The index engine produces reproducible daily values from a documented basket and weights.
- The dashboard displays the index and major analytics without manual database edits.
- The API returns the same values used by the dashboard.
- The 30-day validation/back-test is reproducible and transparently labelled.
- The project is deployed or can be demonstrated locally from documented steps.

- The README explains data sources, ethics/compliance, methodology, limitations and setup.

Recommended hackathon positioning: Present this as an experimental, scalable prototype for high-frequency airfare price measurement. The strongest demonstration is not the number of websites scraped; it is the complete and reproducible chain from permitted price observations to cleaned data, weighted index, validation and policy-ready API.